

Approximation Algorithm for Maximum Cut

Maximum Cut

Consider an undirected graph $G = (V, E)$ with positive edge weights $w(e)$ ($e \in E$). Set $n = |V|$ and $m = |E|$. The Maximum Cut problem (Maxcut) is to find a partition of the vertices with the largest total weight of the edges “crossing” the partition, i.e., maximising the value of

$$c(A) = \sum_{(u,v) \in E, u \in A, v \notin A} w(u, v)$$

over all $A \subseteq V$.

Maxcut is NP-hard, so we have little hope of writing an algorithm that solves arbitrary Maxcut instances optimally.

Algorithm R

Consider the following simple randomized algorithm (call it Algorithm R): Let A be a *random subset* of V constructed by flipping a coin $r(v) \in \{0, 1\}$ for every vertex $v \in V$ and setting $v \in A$ if and only if $r(v) = 1$.

Inputs

The data directory contains two input instances:

pw09_100.9.txt: A random instance with $|V| = 100$ and $|E| = 4455$.

The best cut in this instance is 13658.¹

matching_1000.txt: A disjoint union of 500 edges with unit weight.

The input format is straightforward: the first line contains n and m ; every following line describes an edge in the format first vertex, second vertex, weight. All weights are integers, vertices are numbered $1, 2, \dots, n$.

Algorithm S

Consider the following simple greedy swapping algorithm (call it Algorithm S): Let all the vertices be outside of A to begin with. A vertex can be swapped, which means that if it's outside of A its moved into A and if its inside A its moved out of A . Pick the first vertex you can find that increases your cut if swapped. Swap this vertex, and continue doing so until no vertex increases the cut if swapped, eg you find a local maxima.

¹ A. Wiese, Big Mac Library - A collection of Max-Cut and quadratic 0-1 programming instances of medium size, 2007.

Algorithm RS

Consider the following simple randomized swapping algorithm (call it Algorithm RS): Combine Algorithm R and S by instead of placing all vertices outside of A to begin with in S , place the vertices according to the output of R . Then proceed with the swapping part of S .

Deliverables

1. Implement algorithm R , S and RS and run it on the dataset provided in the data directory. Use whatever programming language and libraries you want, but make sure that your code is short and crisp; Each of the algorithms should not be much more than 20 lines. Reading input + scoring should also be small. Making all the code together less than 100 lines is a suitable goal. Attach a printout of the code to the report.
2. Fill out the report on the next page; you can just use the $\text{\LaTeX}$ code if you want.

Maxcut Lab Report

by Jonathan Andreasson

Running time

Algorithm R	Algorithm S	Algorithm RS
$O(n)$	$O(n + m)$	$O(n + m)$

Randomness

Algorithm R uses n random bits

Solution quality

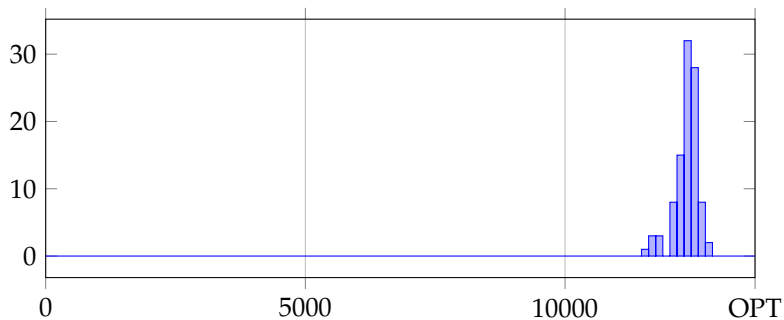
Experiments.

1. For the input file pw09_100.9.txt with $t = 100$ runs, we found for each algorithm, the average cutsize $Avg(C)$ and the maximum cutsize $Max(C)$:

Algorithm	$Avg(C)$	$Max(C)$
R	12323	12636
S	12620	12620
RS	13216	13419

The optimum was given to us as $OPT = 13658$.

The distribution of cutsizes for Algorithm R looks as follows:



The distribution of cutsizes for Algorithm RS looks as follows:

2. For the input file matching_1000.txt $OPT = 500$

Analysis of performance guarantee Clearly, Algorithm R performs quite badly on input matching_1000.txt. We will show that it can perform *no worse* than that, i.e., we will establish that in expectation, the cutsize C satisfies $C \geq 0.5 \cdot OPT$.

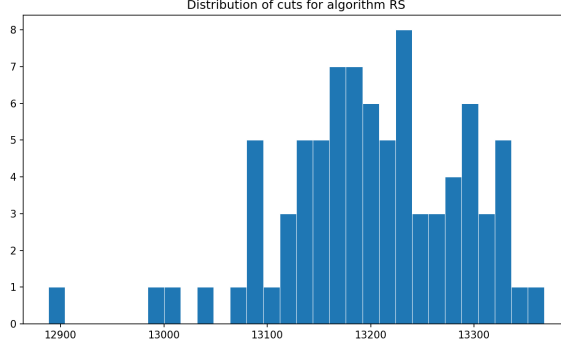


Figure 1: Distribution of cuts for RS

We will view C as a random variable that gives the size of the cut defined by the random choices. Let W denote the total weight of the edges of G , i.e.,

$$W = \sum_{e \in E} w(e).$$

Then,

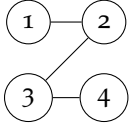
$$E[C] = \frac{1}{2} \cdot W. \quad (1)$$

To see this, define the indicator random variable X_{uv} for every edge $uv \in E$ as follows. Set $X_{uv} = 1$ if uv crosses the cut, i.e., $u \in A$ and $v \notin A$ or $u \notin A$ and $v \in A$. Otherwise, $X_{uv} = 0$.

Then, $\Pr(X_{uv} = 1) = 1/2$ since if we fix $u \in A$, the probability that $v \notin A$ is $1/2$. Now, $E[C] = \sum \Pr(X_{uv} = 1)w(uv)$. Finally, by linearity of expectation, we have $E[C] \geq 1/2 \cdot \text{OPT}$ because clearly $\text{OPT} \leq W$.

Algorithms S and RS perform very well on input `matching_1000.txt`. In fact both algorithms always find the optimum. The reason for this is that for a disjoint union of edges the algorithms always include one endpoint and exclude the other as this maximises the cut and ever including both would decrease it or not increase it.

However Algorithm RS does not always find the optimum even for all bipartite graphs. An example of this scenario is the graph G ,



Where the weight of the edges are

u	v	weight
1	2	2
2	3	3
3	4	2

Here if nodes 2,3 are inside A , Algorithm RS gets stuck in a local

maxima of size 3. The global maxima has size 4, with nodes 1 and 3 inside A .

Perspective

To establish that Maxcut is NP-hard one reduces from NAE-Sat, a reduction that can be found in many places² Recall that the related problem *Minimum Cut* is easy because of the max flow–min cut theorem. A moment’s thought should convince you that as soon as negative weights are allowed, the two problems are the same (and both are hard). Algorithm R doesn’t work at all for negative weights.

Algorithm R is a classical randomised approximation algorithm, its origins seem to be shrouded in the mists of time. The *deterministic* algorithm of Sahni and Gonzales³ can be viewed as a derandomisation of R using the *method of conditional expectations*. These algorithms were best known until the breakthrough result of Goemans and Williamson,⁴ which improved the approximation factor to 0.87856. Håstad has shown that no algorithm can approximate the maxcut better than $16/17 \sim 0.941176$ unless P equals NP. Khot has shown that the Goemans–Williamson bound is essentially optimal under the *Unique Games Conjecture*.

² C. Moore and S. Mertens, *The Nature of Computation*, Oxford University Press, 2011, p. 146.

³ S. Sahni and T. Gonzalez. P-complete approximation problems. *J. Assoc. Comput. Mach.*, 23(3):555–565, 1976.

⁴ M. X. Goemans and D. P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *J. Assoc. Comput. Mach.*, 42(6):1115–1145, 1995.

Printout of code

```

import sys
from random import randint
import numpy as np
# np.random.seed(4)
def read_input(filename):
    with open(filename) as f:
        n, m = map(int, f.readline().split())

        adj = [[] for _ in range(n + 1)]
        for _ in range(m):
            u, v, w = map(int, f.readline().split())
            adj[u].append((v, w))
            adj[v].append((u, w))
        return adj, n, m

def alg_r(adj,n,m):
    r = [np.random.randint(0, 2) for _ in range(n + 1)]
    return r

def alg_s(adj,n,m):
    rs = [0 for _ in range(n+1)]
    same = 0
    diff = 0
    for u in range(1,n+1):
        same = 0
        diff = 0
        for v, w in adj[u]:

            if rs[u] == rs[v]:
                same += w
            else:
                diff += w
        if same-diff > 0:
            rs[u] = 0 if rs[u]==1 else 1
    return rs

def alg_rs(adj,n,m):
    rs = [np.random.randint(0, 2) for _ in range(n + 1)]
    for u in range(1,n+1):
        same = 0
        diff = 0

```

```

    for v, w in adj[u]:
        if rs[u] == rs[v]:
            same += w
        else:
            diff += w
    if same-diff > 0:
        rs[u] = 0 if rs[u]==1 else 1
    return rs

def evaluate(adj, n, m, r):
    cut = 0
    for u in range(1, n + 1):
        for v, w in adj[u]:
            if r[u] != r[v]:
                cut += w
    return cut // 2

if __name__ == '__main__':
    filename1 = 'pw09_100.9.txt'
    filename2 = 'matching_1000.txt'
    print('for ', filename1, ':')
    adj, n, m = read_input(filename1)
    print('    alg_r: ', evaluate(adj, n, m, alg_r(adj, n, m)))
    print('    alg_s: ', evaluate(adj, n, m, alg_s(adj, n, m)))
    print('    alg_rs: ', evaluate(adj, n, m, alg_rs(adj, n, m)))
    print()
    print('for ', filename2, ': ')
    adj, n, m = read_input(filename2)
    print('    alg_r: ', evaluate(adj, n, m, alg_r(adj, n, m)))
    print('    alg_s: ', evaluate(adj, n, m, alg_s(adj, n, m)))
    print('    alg_rs: ', evaluate(adj, n, m, alg_rs(adj, n, m)))

rs_dict = {}
r_sum = 0
s_sum = 0
rs_sum = 0
m1, m2, m3 = 0, 0, 0
for i in range(100):
    adj, n, m = read_input(filename1)
    if (t1 := evaluate(adj, n, m, alg_r(adj, n, m))) > m1:
        m1 = t1

```



```

    r_sum+= t1
    if (t2 := evaluate(adj, n, m, alg_s(adj,n,m))) >m2:
        m2 = t2
    s_sum+= t2
    if (t3 := evaluate(adj, n, m, alg_rs(adj,n,m))) >m3:
        m3 = t3
    if t3 in rs_dict:
        rs_dict[t3]+=1
    else: rs_dict[t3] = 1
    rs_sum+= t3
print(m1,m2 ,m3)
r_avg = r_sum/100
s_avg = s_sum/100
rs_avg = rs_sum/100
print(r_avg, s_avg, rs_avg)

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 5))
plt.hist(rs_dict, bins=30, edgecolor='white', linewidth=0.5)
plt.title('Distribution of cuts for algorithm RS')
plt.savefig('hist.png')

```